Лабораторная работа №19

Создание и изменение объявления на сервере

О чем лабораторная?

Теперь подключим форму создания объявлению к API. Пользователь выберет фотографию, загрузит ее на сервер и создаст настоящее объявление. После этого изменим это объявление через API.

Главное

Создание объявления состоит из двух последовательных запросов:

1. `POST /upload` загружает изображение и возвращает его URL
2. `POST /items` создает объявление и помещает этот URL картинки в поле `photo_url`

Изменение объявления выполняется отдельным запросом - `PATCH /items/{item_id}`.

Часть 1. Посмотрите нужные операции API

Откройте документацию API и найдите:

- `POST /upload`;
- `POST /items`;
- `PATCH /items/{item_id}`.

Для создания объявления сервер ожидает:

Поле	Значение
`title`	название от 5 до 80 символов
`description`	описание от 10 до 500 символов
`price`	целое число от 0
`deal_type`	будет у нас `sale`, `free` или `exchange`
`category_id`	идентификатор категории
`photo_url`	URL изображения, необязательное поле

Часть 2. Опишите запросы в ApiService

Создайте DTO для создания объявления:

```
public class ItemCreate {
    private String title;
    private String description;
    private Integer price;
```

```
    private String deal_type;
    private int category_id;
    private String photo_url;

    public ItemCreate(String title, String description, Integer price,
                      String dealType, int categoryId, String photoUrl) {
        this.title = title;
        this.description = description;
        this.price = price;
        this.deal_type = dealType;
        this.category_id = categoryId;
        this.photo_url = photoUrl;
    }
}
```

Добавьте в `ApiService` два запроса:

```
@POST("items")
Call<ItemResponse> createItem(@Body ItemCreate request);

@Multipart
@POST("upload")
Call<UploadResponse> uploadImage(@Part MultipartBody.Part file);
```

Модель ответа загрузки содержит адрес файла:

```
public class UploadResponse {
    private String url;

    public String getUrl() {
        return url;
    }
}
```

Часть 3. Загрузите выбранную фотографию

В `CreateItemActivity` уже есть `selectedImageUri`, полученный через системную галерею. Перед созданием объявления превратите его в multipart-файл:

```
private MultipartBody.Part createImagePart(Uri uri) throws IOException {
    InputStream input = getContentResolver().openInputStream(uri);
    ByteArrayOutputStream output = new ByteArrayOutputStream();

    byte[] buffer = new byte[4096];
    int count;
    while (input != null && (count = input.read(buffer)) != -1) {
        output.write(buffer, 0, count);
    }
```

```
    if (input != null) {
        input.close();
    }

    RequestBody body = RequestBody.create(
            output.toByteArray(),
            MediaType.parse("image/*")
    );

    return MultipartBody.Part.createFormData(
            "file",
            "item-image.jpg",
            body
    );
}
```

Часть 4. Создайте объявление

Измените обработчик кнопки "Опубликовать" в `CreateItemActivity`.

Порядок действий:

1. Проверьте название, описание, цену и категорию
2. Если выбрано изображение, вызовите `uploadImage()`
3. Дождитесь ответа и получите `response.body().getUrl()`
4. Создайте объект `ItemCreate`
5. Передайте URL изображения в `photo_url`
6. Вызовите `createItem()`
7. После успешного ответа закройте форму

Схема вложенных запросов выглядит так:

```
private void publishItem(ItemCreate request) {
    api.createItem(request).enqueue(new Callback<ItemResponse>() {
        @Override
        public void onResponse(Call<ItemResponse> call,
                               Response<ItemResponse> response) {
            if (response.isSuccessful() && response.body() != null) {
                Toast.makeText(CreateItemActivity.this,
                        "Объявление создано", Toast.LENGTH_SHORT).show();
                finish();
            } else {
                Toast.makeText(CreateItemActivity.this,
                        "Сервер не принял объявление", Toast.LENGTH_LONG).show();
            }
        }

        @Override
        public void onFailure(Call<ItemResponse> call, Throwable t) {
            Toast.makeText(CreateItemActivity.this,
```

```
                    "Ошибка сети", Toast.LENGTH_LONG).show();
        }
    });
}
```

Вызов загрузки фотографии должен привести к этому методу после получения URL:

```
api.uploadImage(createImagePart(selectedImageUri)).enqueue(
        new Callback<UploadResponse>() {
            @Override
            public void onResponse(
                    Call<UploadResponse> call,
                    Response<UploadResponse> response
            ) {
                if (response.isSuccessful() && response.body() != null) {
                    String photoUrl = response.body().getUrl();
                    ItemCreate request = new ItemCreate(
                            title, description, price,
                            dealType, categoryId, photoUrl
                    );
                    publishItem(request);
                } else {
                    showError("Не удалось загрузить изображение");
                }
            }

            @Override
            public void onFailure(Call<UploadResponse> call, Throwable t) {
                showError("Ошибка сети при загрузке изображения");
            }
        }
);
```

Если `selectedImageUri == null`, сразу создайте `ItemCreate` с последним параметром `null` и вызовите `publishItem()`.

Часть 5. Измените объявление

После создания сервер возвращает объявление с числовым `id`. Передайте этот идентификатор на экран редактирования.

Создайте DTO для изменения:

```
public class ItemUpdate {
    private String title;
    private String description;
    private Integer price;
    private String deal_type;
    private Integer category_id;
    private String photo_url;
```

```
    public ItemUpdate(String title, String description, Integer price,
                      String dealType, Integer categoryId, String photoUrl) {
        this.title = title;
        this.description = description;
        this.price = price;
        this.deal_type = dealType;
        this.category_id = categoryId;
        this.photo_url = photoUrl;
    }
}
```

Добавьте в `ApiService`:

```
@PATCH("items/{item_id}")
Call<ItemResponse> updateItem(
        @Path("item_id") int itemId,
        @Body ItemUpdate request
);
```

На экране редактирования:

1. получите `item_id` из Intent
2. заполните поля текущими данными
3. загрузите новую фотографию, если пользователь ее выбрал
4. создайте `ItemUpdate` с новым URL
5. вызовите `updateItem(itemId, request)`

Часть 6. Самостоятельная работа

Подключите к API форму объявления в своем проекте:

1. добавьте загрузку выбранной фотографии через `POST /upload`
2. реализуйте создание `Item` через `POST /items`
3. добавьте изменение объявления через `PATCH /items/{item_id}`

Требования к итоговой версии

- новое объявление создается через `POST /items`
- фотография отправляется через `POST /upload`
- в объявлении сохраняется URL фотографии
- свое объявление можно изменить через `PATCH /items/{item_id}`
- чужое объявление нельзя изменить
- после создания или изменения экран деталей показывает актуальные данные
- фотография из `photo_url` загружается через Glide

Дополнительно (по желанию)

1. Заблокируйте кнопку отправки на время запросов
2. Добавьте индикатор загрузки
3. Добавьте кнопку "Убрать фото" до отправки формы